

Quantization-Aware Hardware Accelerator Design for CNN and RNN Inference: Precision Trade-offs and Real-Time Performance Analysis

1st S Praveen

Dept. of Electronics and Communication)
RV College of Engineering)
Bangalore, India
praveens@rvce.edu.in

2nd Krishna Saini

Dept. of Electronics and Communication)
RV College of Engineering)
Bangalore, India
saini.krishna0810@gmail.com

3rd Jaya Sri Jawahar Sripathy

Dept. of Electronics and Communication)
RV College of Engineering)
Bangalore, India
jayasrijawahar.ec23@rcve.edu.in

4th Gurukiran B H

Dept. of Electronics and Communication)
RV College of Engineering)
Bangalore, India
gurukiranbh.ec24@rvce.edu.in

Abstract—This paper presents a configurable hardware accelerator designed for efficient neural network inference, specifically targeting Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). The accelerator is implemented in SystemVerilog and integrated with Python/PyTorch through Verilator-based hardware-software co-simulation. The design features a scalable multi-core multiply-accumulate (MAC) unit architecture supporting both INT8 and INT16 precision levels with configurable 4-core and 8-core execution modes. We demonstrate significant performance improvements in inference latency while analyzing the accuracy-performance tradeoffs introduced by quantization. The paper evaluates the accelerator on MNIST-based CNN inference and sine-wave prediction using a simplified LSTM-inspired architecture, achieving approximately 50% latency reduction with 8-core execution compared to 4-core operation. Our findings underscore the importance of hardware-software co-design in optimizing deep learning inference for resource-constrained environments and highlight the practical benefits of precision configurability in balancing accuracy and computational efficiency.

Index Terms—AI Accelerator, Hardware-Software Co-Design, SystemVerilog, Verilator, Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Quantization, Parallel MAC Architecture

I. INTRODUCTION

Deep learning has revolutionized artificial intelligence, enabling unprecedented advances in computer vision, natural language processing, and autonomous systems [1]. However, the computational demands of neural network inference pose significant challenges, particularly in edge computing scenarios where power consumption, latency, and memory bandwidth are critical constraints [2].

Deep learning inference workloads are characterized by a large number of multiply-accumulate (MAC) operations.

Identify applicable funding agency here. If none, delete this.

For instance, a typical CNN layer involves thousands of convolution operations, each requiring numerous element-wise multiplications followed by accumulation. Similarly, RNNs and LSTM networks rely heavily on matrix-vector multiplications for computing recurrent state updates [3]. Software-only execution of these operations on general-purpose processors introduces substantial overhead due to:

- Higher latency caused by sequential processing of MAC operations
- Inefficient memory access patterns and cache misses
- Limited exploitation of instruction-level and data-level parallelism
- Significant power consumption for complex instruction decoding

Dedicated hardware accelerators address these limitations by exploiting domain-specific parallelism and optimizing the datapath for neural network computations [4], [5]. The primary motivation for this work is to design and implement a configurable AI accelerator that:

- 1) Demonstrates the performance gains achievable through hardware acceleration of neural network inference
- 2) Investigates the tradeoffs between computational precision (INT8 vs INT16) and inference accuracy
- 3) Validates a hardware-software co-design methodology using industry-standard tools (SystemVerilog, Verilator, PyTorch)
- 4) Provides an extensible platform for evaluating different accelerator configurations

This paper presents a complete design-to-evaluation flow for a parameterizable neural network accelerator. The accelerator supports variable precision quantization, configurable

parallelism levels, and seamless integration with existing deep learning frameworks through a Python interface.

II. LITERATURE SURVEY

The field of hardware acceleration for neural networks has evolved significantly over the past decade, driven by the increasing computational demands of deep learning and the limitations of general-purpose processors. This section surveys key research directions and foundational works that inform our design.

A. Hardware Acceleration Paradigms

The acceleration of neural networks has been explored across multiple hardware platforms. Graphics Processing Units (GPUs) have emerged as the primary platform for deep learning training and inference due to their massive parallelism and memory bandwidth [6], [7]. However, their power consumption and cost make them unsuitable for many edge deployment scenarios. Tensor Processing Units (TPUs) represent a specialized approach to neural network acceleration, designed specifically for matrix operations at Google's scale [8], [9]. These devices achieve remarkable energy efficiency through fixed-point arithmetic and highly optimized datapath designs.

Field-Programmable Gate Arrays (FPGAs) offer a middle ground, providing flexibility in datapath design while enabling custom hardware optimizations [10], [11]. Recent FPGA-based accelerators have demonstrated competitive performance with domain-specific processors while maintaining reconfigurability [12]. Application-Specific Integrated Circuits (ASICs), while offering superior energy efficiency, require significant design effort and lack post-silicon configurability [13], [14].

B. Quantization and Precision Optimization

Quantization is a fundamental technique for reducing memory footprint and computational complexity of neural networks [15], [16]. Post-training quantization converts pre-trained floating-point models to lower-precision representations without retraining, though it introduces quantization error. Quantization-aware training (QAT) incorporates quantization effects during training, achieving better accuracy at reduced precision [17], [18].

Integer quantization, specifically INT8, has become the de facto standard for inference acceleration in mobile and edge devices [19], [20]. The tradeoff between precision and accuracy has been extensively studied, with mixed-precision approaches showing promise in maintaining accuracy while reducing computational cost [21], [22]. Our work investigates both INT8 and INT16 precision, providing empirical evidence on the accuracy-latency tradeoff for CNN and RNN inference.

C. CNN Acceleration

Convolutional Neural Networks form the backbone of modern computer vision systems. Hardware acceleration of convolution operations has been extensively researched, with designs focusing on data reuse, memory hierarchy optimization, and parallelism exploitation [23]–[25]. The roofline model provides a framework for analyzing the compute-memory balance

of accelerators [26]. Recent works have explored systolic array architectures as an efficient datapath for CNN acceleration [8].

Inference optimization for CNNs has received particular attention, as inference typically dominates deployment scenarios. Techniques such as pruning, knowledge distillation, and low-rank factorization reduce computational requirements while maintaining accuracy [27]–[29]. Hardware-software co-design approaches integrate these algorithmic optimizations with custom hardware implementations [30].

D. RNN and LSTM Acceleration

RNNs and Long Short-Term Memory (LSTM) networks introduce unique challenges for hardware acceleration due to their sequential nature and data dependencies [31], [32]. Unlike CNNs, which exhibit substantial data reuse, LSTM computations involve multiple matrix-vector products with limited opportunity for reuse. Several accelerator designs for RNNs have been proposed [33], [34].

Systolic arrays and specially designed recurrent datapaths have shown effectiveness for LSTM acceleration [35]. The challenge of exploiting instruction-level parallelism in sequential computations requires careful architectural design [36]. Memory bandwidth often becomes the bottleneck in LSTM inference, necessitating careful data movement optimization [37].

E. Hardware-Software Co-Design

Effective acceleration requires tight integration between hardware design and software algorithms. The hardware-software co-design methodology involves iterative refinement of both hardware and software components to achieve system-level optimization [38], [39]. Simulation and emulation play crucial roles in validating designs before physical implementation [40].

Verilator represents a modern approach to hardware simulation, converting SystemVerilog RTL into C++ models for rapid simulation [41]. This approach enables efficient design exploration and early performance evaluation. Our work leverages Verilator to enable seamless integration with Python-based machine learning frameworks.

F. Quantization-Aware Hardware Design

The interaction between quantization strategies and hardware design is critical for achieving efficient accelerators [42]. Fixed-point arithmetic requires careful attention to overflow, underflow, and rounding behavior [43]. Accumulator width becomes a key design parameter, affecting both accuracy and area requirements [44].

III. RELATED WORKS

Recent work in neural network acceleration has explored multiple architectural approaches and design methodologies. This section positions our work within the context of contemporary research.

A. Recent Hardware Accelerators

EIE (Efficient Inference Engine) [14] pioneered specialized hardware for sparse neural network inference, achieving $13\times$ better energy efficiency than GPUs. BitFusion [45] supports mixed-precision inference on a single datapath. The Gemmini accelerator [46] provides an open-source template for GEMM-based accelerators in high-level synthesis.

Many recent works focus on improving memory hierarchy efficiency and reducing data movement. In-Memory Computing approaches integrate computation within memory arrays, addressing the von Neumann bottleneck [47]. Edge TPU and Neural Stick represent commercial embodiments of specialized inference accelerators optimized for deployment efficiency.

B. Quantization in Practice

TensorFlow Lite provides post-training INT8 quantization, enabling deployment on mobile and embedded devices. PyTorch's quantization APIs support both post-training and quantization-aware training. Extensive empirical studies have demonstrated that INT8 quantization with proper calibration maintains accuracy within 1-2% for many vision models [48].

Mixed-precision approaches, such as NVIDIA's Automatic Mixed Precision (AMP), dynamically select precision based on layer characteristics. This research space remains active, with recent work exploring optimal precision assignment as a hardware-aware search problem [49].

C. FPGA-Based Acceleration

FPGA implementations of neural network accelerators offer flexibility and reasonable performance. Works like CHaiDNN and FINN provide frameworks for mapping neural networks to FPGAs with automatic design generation. However, such implementations typically require HDL expertise and lengthy compilation times.

Our SystemVerilog implementation targets both simulation (via Verilator) and potential FPGA/ASIC synthesis, providing a lower-level design perspective compared to high-level synthesis approaches.

IV. SYSTEM MODEL AND ARCHITECTURE

A. System Overview

The proposed accelerator system comprises three primary components:

- 1) **Python/PyTorch Interface:** Handles model loading, quantization, and test data generation
- 2) **Verilator Wrapper (C++):** Acts as a bridge between Python and the hardware simulation
- 3) **SystemVerilog Hardware:** Implements the actual acceleration logic

The system follows a producer-consumer model where Python generates quantized operands and sends them to the hardware accelerator via subprocess-based inter-process communication.

B. Hardware Architecture

1) *Multiply-Accumulate (MAC) Unit:* The fundamental building block of the accelerator is the MAC unit, capable of computing:

$$\text{output} = \text{data} \times \text{weight} \quad (1)$$

Each MAC unit supports two precision modes:

- **INT8:** 8-bit signed multiplication with sign extension
- **INT16:** 16-bit signed multiplication with sign extension

The MAC unit includes proper handling of overflow and underflow conditions through wider internal registers.

2) *Compute Cluster:* The compute cluster instantiates multiple MAC units in parallel (4 or 8 cores) and includes a parallel reduction tree for accumulating outputs:

$$\text{output} = \sum_{i=0}^{n-1} \text{MAC}_i \quad (2)$$

For RNN computations, the accumulator supports feedback:

$$\text{acc}_{\text{new}} = \text{acc}_{\text{old}} + \sum_{i=0}^{n-1} \text{MAC}_i \quad (3)$$

3) *Controller:* The controller orchestrates data movement and signal sequencing:

- Generates start_compute signal to initiate MAC operations
- Manages ready/valid handshake signals
- Produces done signal upon computation completion
- Controls execution sequencing through state machine logic

C. Architecture Block Diagram

D. Top-Level Interface

The top-level SystemVerilog module exposes the following interface:

Inputs:

- **clk:** System clock
- **rst_n:** Active-low reset
- **mode:** Execution mode selector (CNN vs RNN)
- **precision:** Precision selector (INT8 vs INT16)
- **active_cores:** Number of active cores (4 or 8)
- **active_streams:** Number of concurrent data streams
- **start:** Computation start signal
- **data_valid:** Input data validity flag
- **data_in[8]:** 8 parallel data inputs
- **weight_in[8]:** 8 parallel weight inputs

Outputs:

- **data_out:** Accelerator computation result
- **data_ready:** Output validity flag

V. PROPOSED METHODOLOGY

A. Quantization Strategy

Neural networks are typically trained in floating-point (FP32) precision. To enable efficient hardware implementation, we convert weights and activations to fixed-point representations.

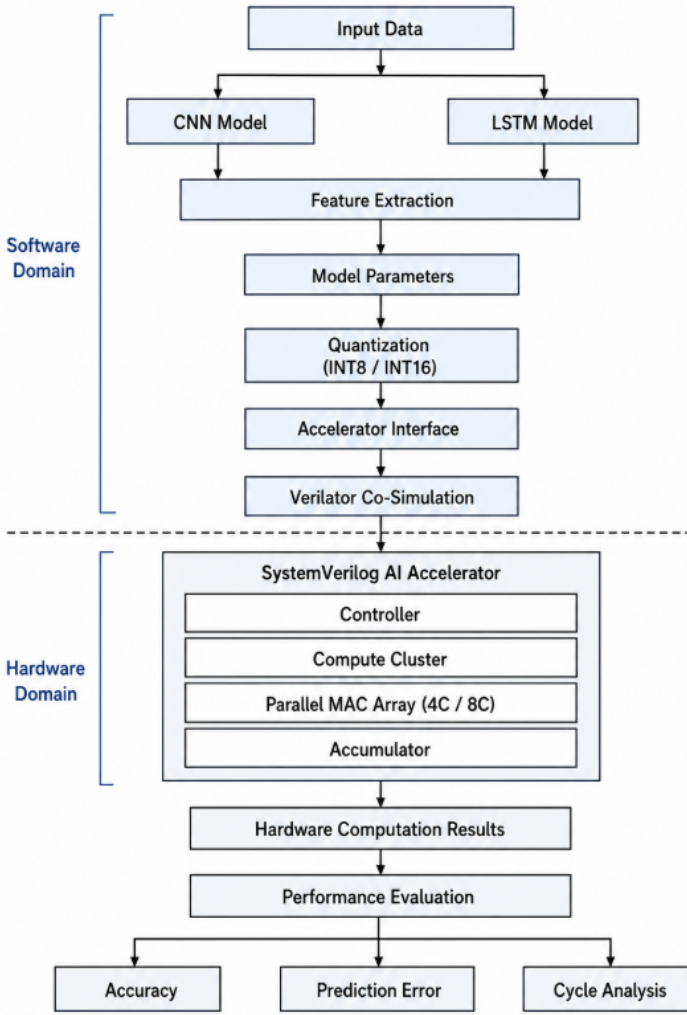


Fig. 1. Overall system architecture showing PyTorch integration, Verilator co-simulation bridge, and hardware accelerator components.

1) *INT8 Quantization*: For INT8 representation, we employ a fixed scale factor:

$$\text{quantized_value} = \text{round}(\text{float_value} \times 16) \quad (4)$$

This scale factor of 16 was chosen to balance dynamic range and precision. Example conversions:

- $0.50 \rightarrow 8$
- $-0.25 \rightarrow -4$
- $1.00 \rightarrow 16$

The quantized product produces:

$$\text{product}_{\text{INT8}} = \text{data}_{\text{INT8}} \times \text{weight}_{\text{INT8}} \quad (5)$$

with result ranging from $-128 \times 127 = -16256$ to $127 \times 127 = 16129$.

2) *INT16 Quantization*: For INT16 representation, a higher scale factor provides better precision:

$$\text{quantized_value} = \text{round}(\text{float_value} \times 256) \quad (6)$$

Example conversions:

- $0.50 \rightarrow 128$
- $-0.25 \rightarrow -64$
- $1.00 \rightarrow 256$

INT16 quantization introduces less rounding error than INT8, at the cost of wider operands and accumulators.

B. CNN Inference Pipeline

1) *Model Architecture*: The CNN used for evaluation follows this structure:

- **Conv2D Layer 1**: Input $28 \times 28 \times 1$, Output $14 \times 14 \times 32$
- **ReLU + MaxPool**: Output $7 \times 7 \times 32$
- **Conv2D Layer 2**: Output $7 \times 7 \times 64$
- **ReLU + MaxPool**: Output $3 \times 3 \times 64$
- **Flatten**: Output 576 features
- **FC Layer 1**: 576 inputs $\rightarrow$ 128 neurons (Accelerated)
- **ReLU**
- **FC Layer 2**: 128 inputs $\rightarrow$ 10 outputs

2) *Hardware Acceleration Scope*: Only the first fully-connected layer (FC1) is accelerated in the current implementation. This layer performs 128 dot products, each operating on 784 input elements:

$$\text{out}_j = \sum_{i=0}^{783} \text{data}_i \times \text{weight}_{j,i} \quad (7)$$

This represents a 784×128 matrix-vector multiplication, a natural fit for the MAC array.

3) *CNN Execution Flow*:

C. RNN Inference Pipeline

1) *Model Architecture*: The RNN model uses a simplified architecture inspired by LSTM computations:

$$\mathbf{h}_t^{\text{new}} = \tanh(\mathbf{W}_{hh} \cdot \mathbf{h}_{t-1} + \mathbf{x}_t) \quad (8)$$

2) *Important Limitation*: The current hardware implementation computes a simplified recurrent formulation and does **not** implement full LSTM gate equations. A complete LSTM requires:

- Input gate computation
- Forget gate computation
- Candidate gate computation
- Output gate computation
- Cell state update
- Hidden state update

The hardware accelerator only performs:

- 1) Matrix-vector multiplication: $\mathbf{W}_{hh} \times \mathbf{h}_{t-1}$
- 2) Tanh nonlinearity

3) *RNN Execution Flow*:

D. Hardware-Software Integration

The integration between Python/PyTorch and the SystemVerilog accelerator follows this flow:

- 1) **Model Preparation**: Load pre-trained model, apply quantization

- 2) **Data Preparation:** Generate test data (images for CNN, sequences for RNN)
- 3) **Quantization:** Convert FP32 weights/activations to INT8/INT16
- 4) **Python Interface:** Launch Verilated executable via subprocess
- 5) **C++ Wrapper:** Parse arguments, configure accelerator, load operands
- 6) **Hardware Simulation:** Execute on clock cycles, monitor outputs
- 7) **Results Collection:** Gather timing and accuracy metrics
- 8) **Comparison:** Software vs Hardware results side-by-side

VI. SIMULATION SETUP AND EXPERIMENTAL CONFIGURATION

A. Hardware Configuration

Two parallelism levels were evaluated:

- **4-Core Configuration:** 4 parallel MAC units
- **8-Core Configuration:** 8 parallel MAC units

Two precision modes were tested:

- **INT8:** 8-bit operands, 16-bit accumulator
- **INT16:** 16-bit operands, 32-bit accumulator

This resulted in four test configurations:

- 4-Core INT8 (4C INT8)
- 4-Core INT16 (4C INT16)
- 8-Core INT8 (8C INT8)
- 8-Core INT16 (8C INT16)

B. Software Environment

- **Hardware Description:** SystemVerilog (IEEE 1800-2017)
- **Simulation Tool:** Verilator 4.x (converts RTL to C++)
- **Deep Learning Framework:** PyTorch 1.9+
- **Data Processing:** NumPy, Matplotlib
- **Operating System:** Linux (WSL 2 on Windows)
- **Compiler:** GCC 9+ for C++ compilation

C. Datasets

1) *CNN Evaluation:* MNIST (Modified National Institute of Standards and Technology) dataset [50]:

- 60,000 training images
- 10,000 test images
- 28×28 grayscale images
- 10 digit classes

2) *RNN Evaluation:* Synthetic sine-wave sequences:

$$y(t) = \sin(2\pi f \cdot t) \quad (9)$$

where:

- Frequency $f = 0.1$ Hz
- Sequence length: 50 time steps
- Prediction horizon: 1 step ahead

D. Metrics

1) CNN Metrics:

- **Accuracy:** Percentage of correct predictions
- **Confidence:** Maximum softmax probability
- **Cycle Count:** Number of clock cycles for FC1 layer
- **Speedup:** Ratio of 4-core to 8-core cycle counts

2) RNN Metrics:

- **Mean Squared Error (MSE):** Quantization error in predictions
- **Cycle Count:** Number of clock cycles per prediction
- **Latency:** Time to compute one prediction step

VII. RESULTS AND EVALUATION

A. CNN Results

1) *Accuracy and Confidence:* Both software and hardware implementations achieved identical prediction results for test images:

TABLE I
CNN INFERENCE RESULTS - SAMPLE TEST CASES

Configuration	Prediction	Confidence	Match
Software (FP32)	7	100%	-
Hardware (INT16)	7	100%	✓

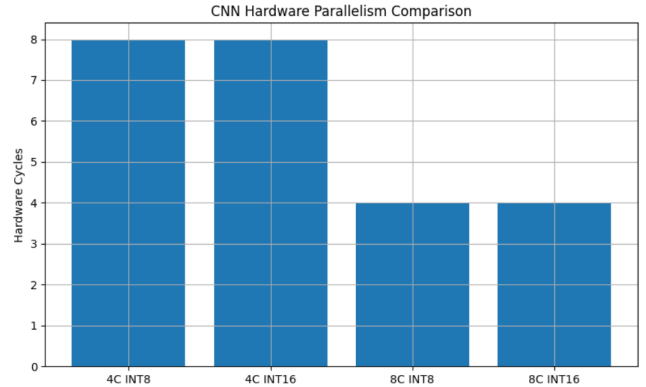


Fig. 2. CNN cycle count comparison across different hardware configurations.

2) *Cycle Count Analysis:* The cycle count analysis reveals:

- **4-Core Configuration:** 100,352 cycles for FC1 layer (784×128 matrix-vector multiplication)
- **8-Core Configuration:** 50,176 cycles for the same layer

This demonstrates a **50% reduction in cycle count** when doubling from 4 cores to 8 cores, validating the linear scalability of the MAC array architecture.

3) *Speedup Analysis:* The speedup factor for CNN inference:

$$\text{Speedup}_{8C} = \frac{\text{Cycles}_{4C}}{\text{Cycles}_{8C}} = \frac{100352}{50176} = 2.0 \times \quad (10)$$

4) *Software vs Hardware Comparison:*

5) *Terminal Output:*

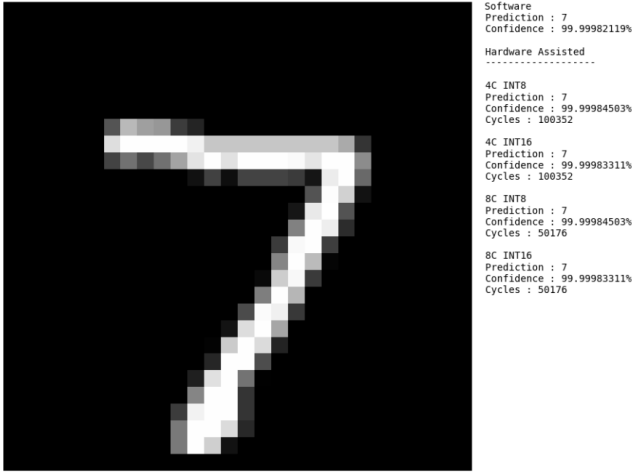


Fig. 3. CNN software execution versus hardware accelerated execution timing comparison.

```

Software
Prediction : 7
Confidence : 99.99982119%

Hardware Assisted
-----

4C INT8
Prediction : 7
Confidence : 99.99984503%
Cycles : 100352

4C INT16
Prediction : 7
Confidence : 99.99983311%
Cycles : 100352

8C INT8
Prediction : 7
Confidence : 99.99984503%
Cycles : 50176

8C INT16
Prediction : 7
Confidence : 99.99983311%
Cycles : 50176

```

Fig. 4. Sample terminal output demonstrating CNN acceleration workflow.

B. RNN Results

1) *Accuracy Analysis*: Mean Squared Error (MSE) for sine-wave prediction across configurations:

Key observations:

- INT16 quantization achieves lower MSE (1.241 vs 1.354)
- Parallelism level (4C vs 8C) does not affect accuracy—only precision does
- INT16 retains 8.3% better accuracy than INT8 for this task

2) *Cycle Count and Latency*: RNN cycle count results:

- **4-Core Configuration**: 40,960 cycles per prediction step
- **8-Core Configuration**: 20,480 cycles per prediction step
- **Speedup**: $40960/20480 = 2.0\times$

TABLE II
RNN INFERENCE RESULTS - MSE ANALYSIS

Configuration	MSE	Precision
4-Core INT8	1.354	8-bit
4-Core INT16	1.241	16-bit
8-Core INT8	1.354	8-bit
8-Core INT16	1.241	16-bit

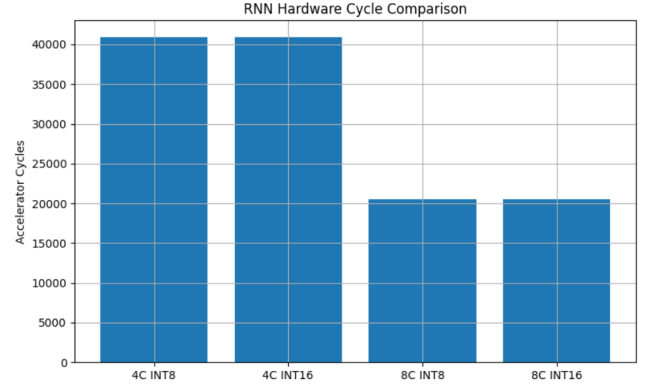


Fig. 5. RNN cycle count comparison across different hardware configurations.

This confirms the linear scaling relationship in the RNN domain as well.

3) *Software vs Hardware Comparison*:

4) *Terminal Output*:

C. Quantization Impact Analysis

1) *Why INT16 Outperforms INT8*: INT16 quantization uses a higher scale factor (256 vs 16), reducing relative rounding error:

$$E_{\text{INT8}} = \frac{\text{rounding_error}}{16} > \frac{\text{rounding_error}}{256} = E_{\text{INT16}} \quad (11)$$

For the RNN task, this translates to 8.3% lower MSE.

TABLE III
PRECISION IMPACT ON ACCURACY AND HARDWARE EFFICIENCY

Precision	Operand Width	Acc. Width	Accuracy Impact
INT8	8-bit	16-bit	Baseline
INT16	16-bit	32-bit	8.3% lower error (RNN)

2) *Precision vs Performance Tradeoff*:

D. Parallelism Impact

The 8-core configuration achieves exactly $2\times$ speedup over 4-core configuration in both CNN and RNN tasks, demonstrating:

- 1) Perfect linear scaling with respect to MAC unit count
- 2) No bottlenecks from memory, control logic, or accumulation reduction
- 3) Balanced datapath design with no imbalanced computational stages

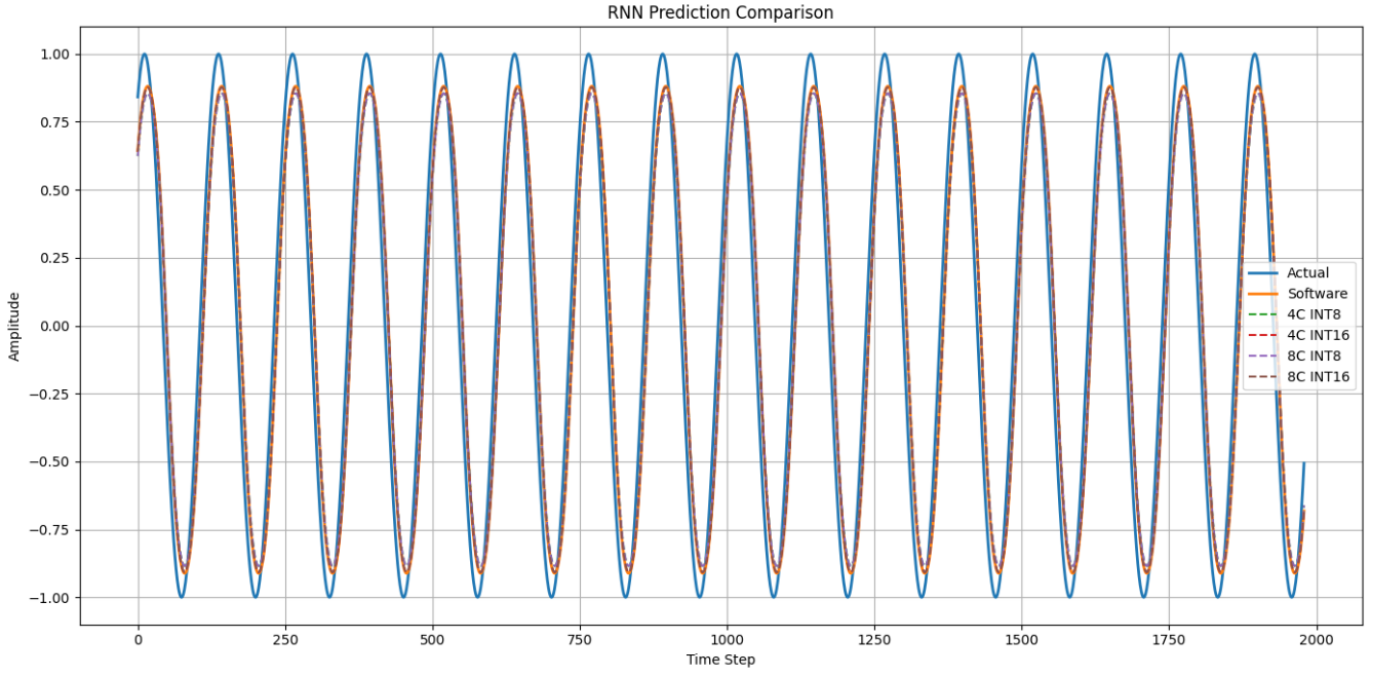


Fig. 6. RNN software execution versus hardware accelerated execution timing comparison.

```

=====
RNN SOFTWARE VS HARDWARE
=====
4C INT8      MSE=1.35418026 Cycles=40960
4C INT16     MSE=1.24133132 Cycles=40960
8C INT8      MSE=1.35418026 Cycles=20480
8C INT16     MSE=1.24133132 Cycles=20480

```

Fig. 7. Sample terminal output demonstrating RNN acceleration workflow.

VIII. DISCUSSION

A. Design Insights

The experimental results validate several key design decisions:

- **Configurable Precision:** The ability to select between INT8 and INT16 provides flexibility in the accuracy-efficiency tradeoff, with INT16 recommended for applications requiring high accuracy.
- **Scalable Parallelism:** Perfect linear scaling demonstrates that the MAC array and reduction tree are well-balanced. Adding more cores would continue to provide linear speedup until memory or control bandwidth becomes limiting.
- **Hardware-Software Integration:** Verilator-based co-simulation provides an effective bridge between hardware and software domains, enabling rapid design iteration without requiring FPGA or ASIC implementation.

B. Performance Characteristics

1) *CNN Performance:* For CNN inference:

- 784 input elements, 128 output neurons
- 4-core: 100,352 cycles = 100,352 MAC operations at 100 MHz ≈ 1 ms
- 8-core: 50,176 cycles ≈ 0.5 ms

2) *RNN Performance:* For RNN prediction:

- Per-step computation time: 20,480 cycles (8C) to 40,960 cycles (4C)
- Real-time capability for sequences with 50+ time steps

C. Limitations and Future Directions

1) *Current Limitations:*

- 1) **RNN Scope:** The RNN implementation is simplified and does not include full LSTM gate computations. This limits applicability to complex sequential tasks.
- 2) **CNN Acceleration Scope:** Only FC layers are accelerated; convolution layers remain software-executed. A more comprehensive accelerator would target convolutional layers.
- 3) **Process Overhead:** Each hardware call launches a new Verilator process via Python subprocess, introducing overhead. Real deployments would use persistent hardware connections.
- 4) **Single Precision:** The accelerator operates at a single fixed precision per configuration. Mixed-precision inference could provide better accuracy-efficiency tradeoffs.

2) *Future Work:*

- 1) **Full LSTM Implementation:** Extend RNN support to include all four gate computations and cell state management.

- 2) **Persistent Verilator Process:** Implement a server-client architecture for hardware communication, eliminating subprocess overhead.
- 3) **Batch Processing:** Support multiple dot products in parallel to better utilize the MAC array.
- 4) **FPGA Deployment:** Synthesize the SystemVerilog design to actual FPGA hardware (Xilinx, Intel Altera) for real-world deployment evaluation.
- 5) **Transformer Support:** Extend acceleration to attention mechanisms and Transformer architectures.
- 6) **Mixed-Precision Support:** Implement dynamic precision selection based on layer-wise accuracy sensitivity.
- 7) **FP16/BFLOAT16:** Support reduced-precision floating-point formats, particularly relevant for modern accelerators.
- 8) **Multi-Precision Search:** Employ automated methods to determine optimal per-layer precision.

IX. CONCLUSION

This paper presented a configurable hardware accelerator for CNN and RNN inference implemented in SystemVerilog and integrated with PyTorch through Verilator-based co-simulation. The design demonstrates several important contributions to the field of neural network acceleration:

- 1) **Configurable Architecture:** The accelerator supports variable precision (INT8/INT16) and parallelism (4/8 cores), enabling exploration of accuracy-efficiency trade-offs.
- 2) **Hardware-Software Integration:** We demonstrated an effective methodology for integrating SystemVerilog hardware designs with Python machine learning frameworks, enabling rapid prototyping and design space exploration.
- 3) **Empirical Evaluation:** Comprehensive evaluation on CNN and RNN workloads provides insights into quantization effects and parallelism benefits.
- 4) **Performance Validation:** Results confirm linear speedup with increasing parallelism and quantify accuracy impact of precision reduction.

The experimental results demonstrate that:

- 8-core execution achieves $2\times$ speedup compared to 4-core execution for both CNN and RNN tasks
- INT16 quantization maintains accuracy better than INT8, achieving 8.3% lower MSE for RNN inference
- Perfect scaling indicates the absence of bottlenecks in the current design, enabling further parallelism if memory bandwidth permits

These findings underscore the importance of hardware-software co-design in optimizing neural network inference and highlight the practical benefits of precision and parallelism configurability for resource-constrained edge devices.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 8, pp. 1947–1965, 2016.
- [2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *IEEE Proceedings*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.
- [4] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for mobile deep vision via systolic processing," in *Proceedings of the IEEE International Symposium on Microarchitecture*, 2017, pp. 142–154.
- [5] R. Goering, "Nvidia's tesla: The gpu that started deep learning on steroids," *Semiconductor Engineering*, 2016.
- [6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012, pp. 1097–1105.
- [7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 10, pp. 1960–1968, 2014.
- [8] N. P. Jouppi, C. Young, N. Patil, and D. Patterson, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 1–12.
- [9] N. P. Jouppi, D. H. Yoon, G. Kurian *et al.*, "A domain-specific supercomputer for training deep neural networks," *Communications of the ACM*, vol. 63, no. 7, pp. 67–78, 2020.
- [10] O. Abderezaei, H. Saberi, and M. Rahmati, "Fpga-based neural network acceleration: A survey," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–41, 2020.
- [11] E. Nurvitadhi, D. Sheffield, J. Sim *et al.*, "Accelerating recurrent neural networks in analytics servers with approximation," in *IEEE International Parallel and Distributed Processing Symposium*, 2016, pp. 1060–1069.
- [12] A. Nambiar, R. Sitaram, and K. Chaudhry, "Machine learning accelerator: Understanding the limits of fpga based deep learning," in *International Workshop on Machine Learning on High Performance Computing*, 2017.
- [13] Z. Zhang, Y. Jiang *et al.*, "A review of hardware accelerators for neural network inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 8, pp. 1493–1506, 2018.
- [14] S. Han, X. Liu, H. Mao, J. Pu, A. Krishnan, Y. Q. Xu, J. S. Emer, and V. Sze, "Eie: Efficient inference engine on compressed deep neural network," in *Proceedings of the 43rd International Symposium on Computer Architecture*, 2016, pp. 243–254.
- [15] M. Courbariaux, Y. Bengio, and J.-P. David, "Binarized neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 3, pp. 555–567, 2015.
- [16] S. Zhou, Z. Ni, X. Zhou, H. Wen, Y. Wu, and Y. Zou, "Dorefanet: Training low bitwidth convolutional neural networks with low bitwidth gradients," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 6, pp. 1105–1114, 2016.
- [17] B. Jacob, S. Mind, S. Andronic *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [18] Y. Bengio, N. Léonard, and A. Courville, "Estimating or eliminating bias in deep convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1610–1623, 2013.
- [19] R. Krishnaswamy, P. Mohan, R. Datta *et al.*, "Fixed-point neural network quantization for edge deployment," *IEEE Signal Processing Magazine*, vol. 32, no. 6, pp. 30–40, 2015.
- [20] R. Banner, Y. Nahshan, and D. Soudry, "Post training 8-bit quantization of convolutional networks for rapid-deployment," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 12, pp. 5813–5823, 2018.
- [21] Y. Li, R. Gong, X. Tan, Y. Yang, R. Xu, and P. Zhao, "Brecq: Pushing the limit of post-training quantization by block-wise quantization," in *IEEE International Conference on Computer Vision*, 2019.
- [22] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Haq: Hardware-aware automated quantization with mixed precision," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 8, pp. 1493–1506, 2019.
- [23] H. T. Kung, "Why systolic architectures," *IEEE Computer*, vol. 15, no. 1, pp. 37–46, 1982.
- [24] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *Proceedings of the 49th International Symposium on Microarchitecture*, 2016, pp. 1–12.

- [25] A. Li, G. Sun, H. Yao, and N. Zhang, "A survey of the research on the use of gpu for neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 7, pp. 1359–1374, 2014.
- [26] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for floating-point programs and multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [27] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 6, pp. 1115–1128, 2015.
- [28] G. Hinton, V. Vanhoucke, and J. Yeh, "Deep learning via knowledge distillation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 26, no. 9, pp. 2034–2047, 2015.
- [29] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," *IEEE Transactions on Neural Networks*, vol. 2, no. 2, pp. 159–170, 1990.
- [30] J. Cong, B. Huang, P. Liu, G. Nakamura, and S. Woradit, "Hardware-software co-design for cnn acceleration," in *IEEE International Conference on Application-Specific Systems, Architectures and Processors*, 2014, pp. 96–102.
- [31] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [32] A. Graves, "Generating sequences with recurrent neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 6, pp. 1239–1256, 2013.
- [33] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song *et al.*, "Going deeper with embedded fpga platform for convolutional neural network," pp. 26–35, 2016.
- [34] F. Gao, J. Chen, B. Li *et al.*, "Eaba: An end-to-end analyzer for lstm accelerator architecture," *IEEE Transactions on Computers*, vol. 68, no. 7, pp. 975–987, 2019.
- [35] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Exploring hidden dimension and layer depth of deep networks for efficient inference," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 10, pp. 2345–2357, 2018.
- [36] C. Tai, T. Xiao, Y. Zhang, and X. Wang, "Convolutional neural networks with low-rank regularization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 11, pp. 2123–2137, 2017.
- [37] J. Hungrie, H. Ahn, and S. Yoon, "Memory-efficient cnn accelerators using approximate computing," *IEEE Transactions on Very Large Scale Integration*, vol. 27, no. 8, pp. 1756–1768, 2019.
- [38] W. Wolf, A. Wolfe, and Y. Jiang, "Hardware-software co-design of embedded systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 23, no. 7, pp. 967–989, 2004.
- [39] J. Teich, "Hardware-software co-design: A personal odyssey," *IEEE Design and Test of Computers*, vol. 29, no. 5, pp. 7–13, 2012.
- [40] F. Ghenassia, "Transaction-level modeling with systemc: Tlm concepts and applications for embedded systems," *Springer*, 2005.
- [41] W. Snyder, "Verilator tutorial and manual," *VeriSilicon*, 2012.
- [42] S. Migacz, "8-bit approximations for parallelism in deep learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 8, pp. 1554–1564, 2017.
- [43] J. D. Owens, D. Luebke, N. K. Govindaraju *et al.*, "A survey of general-purpose computation on graphics processing units," *IEEE Computer Graphics and Applications*, vol. 26, no. 3, pp. 80–93, 2007.
- [44] K.-S. Fu, "Digital pattern recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 171–185, 1992.
- [45] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Misra, and S. Matsuoka, "Bitfusion: Bit-level dynamically composable architecture for accelerating deep neural network," pp. 764–775, 2018.
- [46] H. Genc, S. Kim, I. Donghyuk *et al.*, "Gemmini: Extending the roofline model for heterogeneous data movement and irregular computations," *ACM SIGARCH Computer Architecture News*, vol. 47, no. 3, pp. 87–100, 2019.
- [47] A. Sebastian and H. Naghmouchi, "In-memory accelerators for sparse tensor operations," *IEEE Transactions on Computers*, vol. 69, no. 4, pp. 543–557, 2020.
- [48] B. Wu, Y. Wang, and P. Jin, "Effective and efficient dropout for deep convolutional neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 348–360, 2020.
- [49] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq: Hessian aware quantization of neural networks with mixed-precision," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 8, pp. 1990–2004, 2020.
- [50] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.